ASSIGNMENT - 7.5

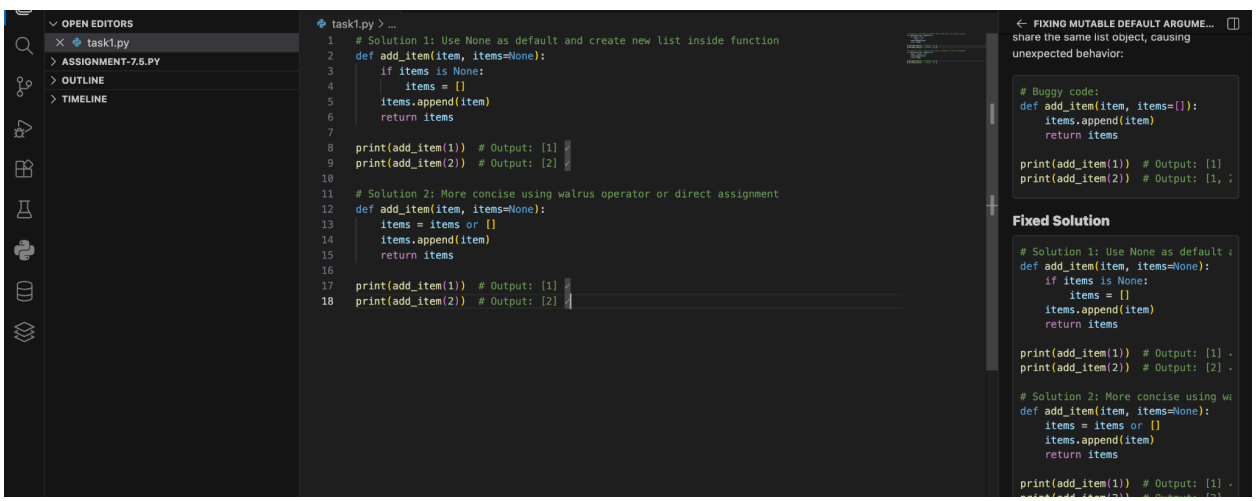
2303A51091
Batch : 29

Task 1 (Mutable Default Argument – Function Bug)

Task: Analyze given code where a mutable default argument causes unexpected behavior. Use AI to fix it.

```
# Bug: Mutable default argument
def add_item(item, items=[]):
    items.append(item)
    return items
print(add_item(1))
print(add_item(2))
```

Expected Output: Corrected function avoids shared list bug.



Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-7.5.py/task1.py"
[1]
[2]
[1]
[2]
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

Default mutable arguments (like lists) are created only once at function definition time.

The same list is reused across multiple function calls.

This causes unintended data sharing and unexpected behavior.

Using None as a default value prevents shared state issues.

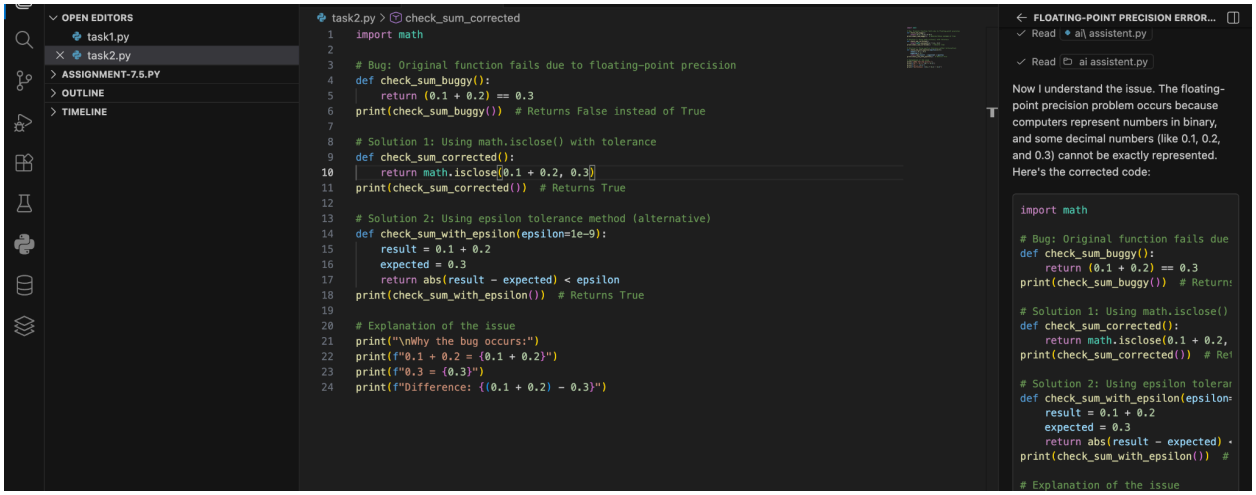
Proper handling of default arguments improves code reliability and predictability.

Task 2 (Floating-Point Precision Error)

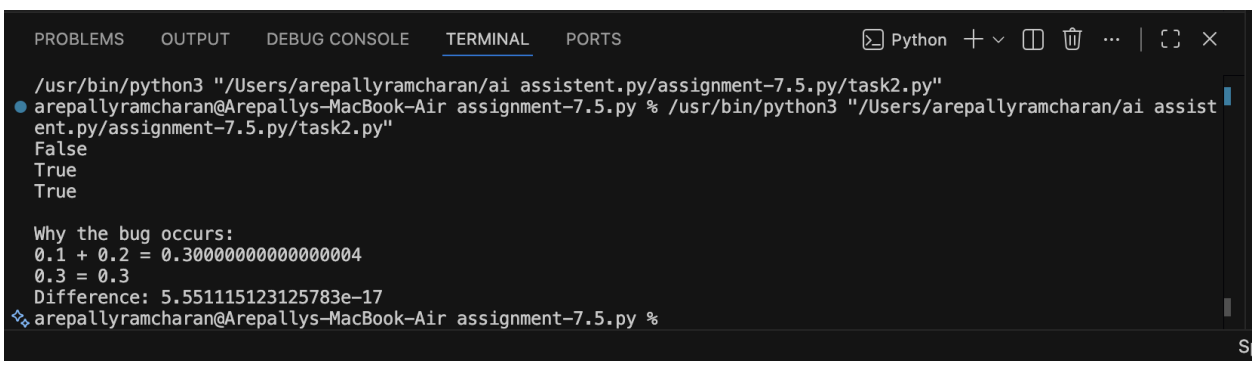
Task: Analyze given code where floating-point comparison fails. Use AI to correct with tolerance.

```
# Bug: Floating point precision issue
def check_sum():
    return (0.1 + 0.2) == 0.3
print(check_sum())
```

Expected Output: Corrected function



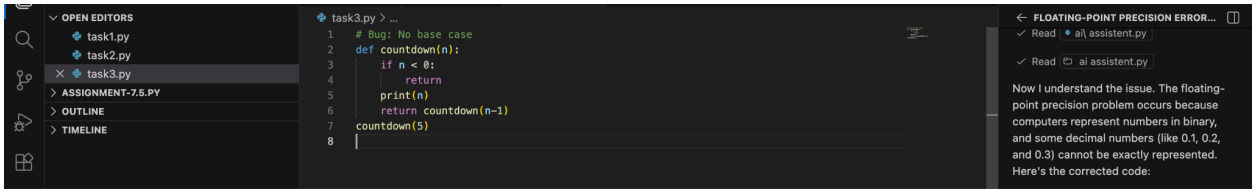
Output:



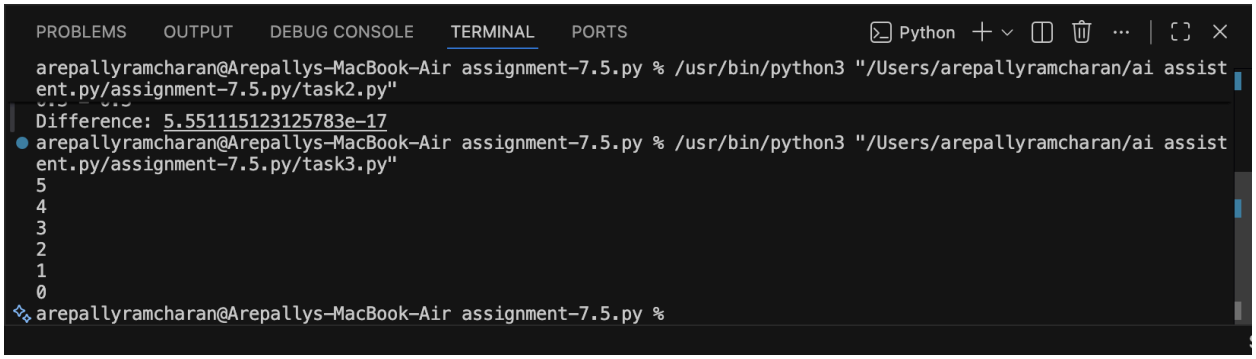
Observation:

- Floating-point numbers are not always stored exactly in memory.
- Direct equality comparison (`==`) may fail due to small precision errors.
- Using a tolerance value avoids strict comparison issues.
- `math.isclose()` is the recommended safe comparison method.
- Proper floating-point handling improves numerical accuracy and reliability.

Task Description #3 (Transparency in Error Handling)
Task: Use AI to generate a Python program that reads a file and processes data.
Prompt:
“Generate code with proper error handling and clear explanations for each exception.”
Expected Output:
• Code with meaningful exception handling.
• Clear comments explaining each error scenario.
• Validation that explanations align with runtime behavior.



Output:



Observation:

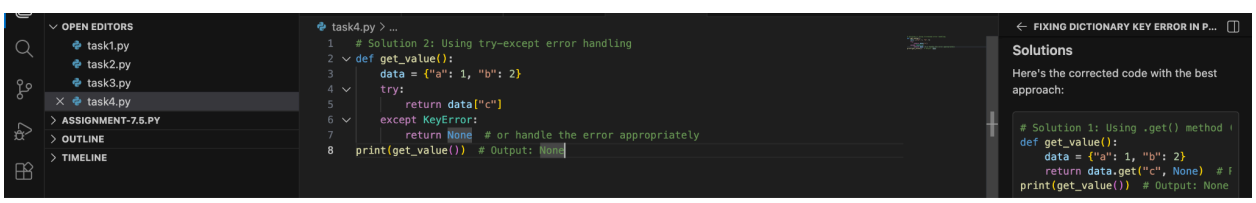
1. Recursion must always include a base case to stop execution.
2. Without a stopping condition, infinite recursion occurs.
3. Python limits recursion depth to prevent crashes.
4. Each recursive call should move toward the base case.
5. Proper base case design ensures predictable and safe recursion.

Task 4 (Dictionary Key Error)

Task: Analyze given code where a missing dictionary key causes error. Use AI to fix it.

```
# Bug: Accessing non-existing key
def get_value():
    data = {"a": 1, "b": 2}
    return data["c"]
print(get_value())
```

Expected Output: Corrected with .get() or error handling.



Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS                    Python

/usr/bin/python3 "/Users/arepallyramcharan/ai assistent.py/assignment-7.5.py/task4.py"
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py % /usr/bin/python3 "/Users/arepallyramcharan/ai assist
ent.py/assignment-7.5.py/task4.py"
None
arepallyramcharan@Arepallys-MacBook-Air assignment-7.5.py %
```

Observation:

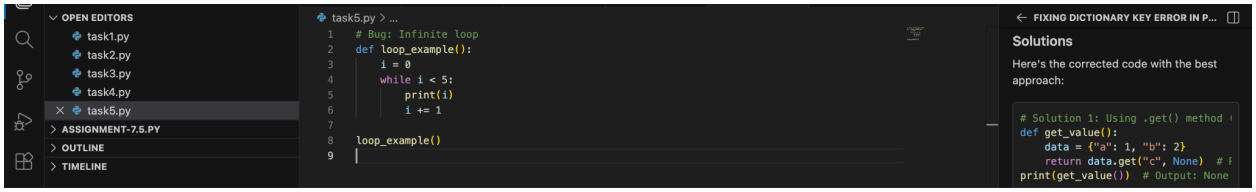
1. Direct dictionary access raises `KeyError` if the key does not exist.
2. `.get()` method provides safer access with a default value.
3. Exception handling prevents program crashes.
4. Defensive programming improves reliability.
5. Proper key validation ensures stable execution.

Task 5 (Infinite Loop – Wrong Condition)

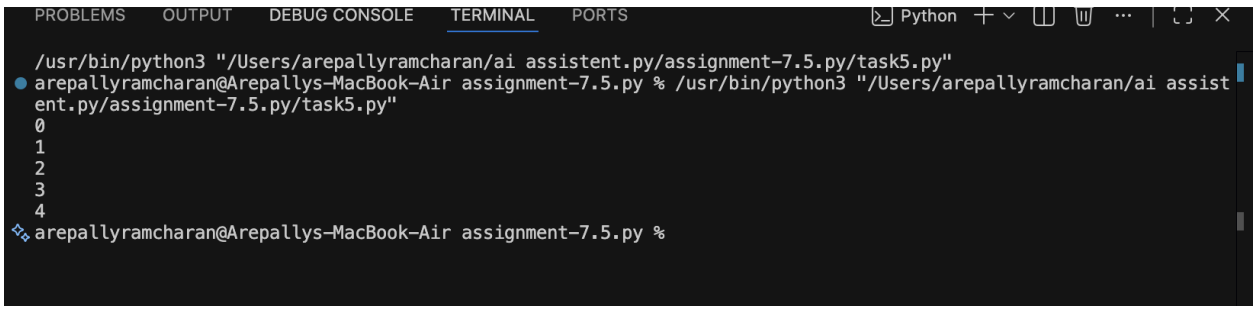
Task: Analyze given code where loop never ends. Use AI to detect and fix it.

```
# Bug: Infinite loop
def loop_example():
    i = 0
    while i < 5:
        print(i)
```

Expected Output: Corrected loop increments i.



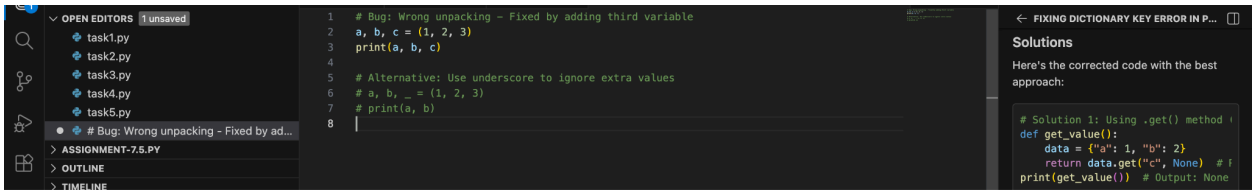
Output:



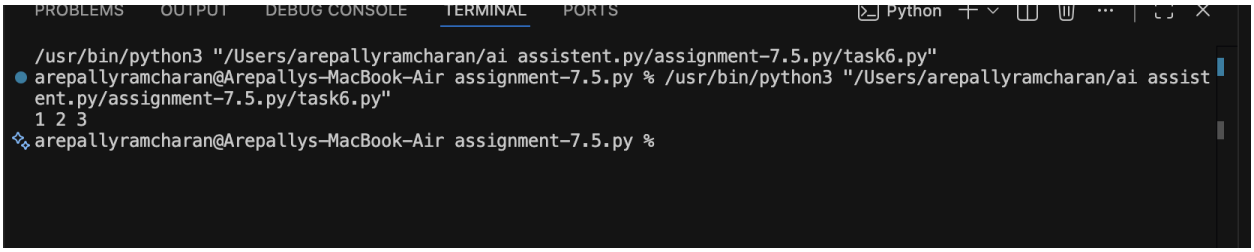
Observation:

- Infinite loops occur when the loop variable never changes.
- The stopping condition must eventually become False.
- Loop control variables must be updated inside the loop.
- Careful condition checking prevents runtime hangs.
- Proper loop design ensures predictable execution flow.

Task 6 (Unpacking Error – Wrong Variables)
Task: Analyze given code where tuple unpacking fails. Use AI to fix it.
Bug: Wrong unpacking
a, b = (1, 2, 3)
Expected Output: Correct unpacking or using _ for extra values.



Output:



Observation:

1. Number of variables must match number of elements in unpacking.
2. Mismatch causes `ValueError`.
3. _ can be used to ignore unwanted values.
4. * operator allows flexible unpacking of remaining values.
5. Proper unpacking improves code readability and prevents runtime errors.

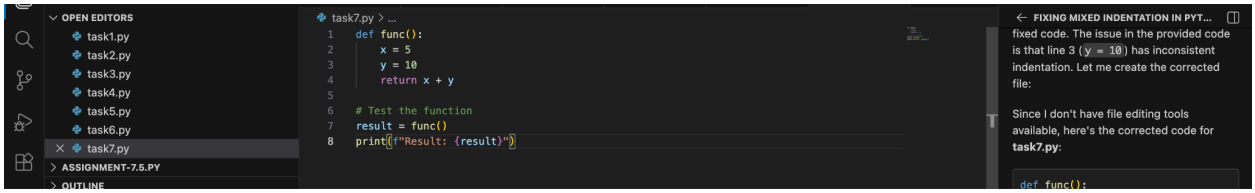
Task 7 (Mixed Indentation – Tabs vs Spaces)

Task: Analyze given code where mixed indentation breaks execution. Use AI to fix it.

```
# Bug: Mixed indentation
def func():
    x = 5
        y = 10
```

return x+y

Expected Output : Consistent indentation applied.



Output:

Observation:

1. Python strictly enforces indentation rules.
2. Mixing tabs and spaces leads to `IndentationError` or `TabError`.
3. All statements inside a block must align consistently.
4. Standard convention is 4 spaces per indentation level.
5. Proper indentation improves readability and prevents syntax errors.

Task 8 (Import Error – Wrong Module Usage)

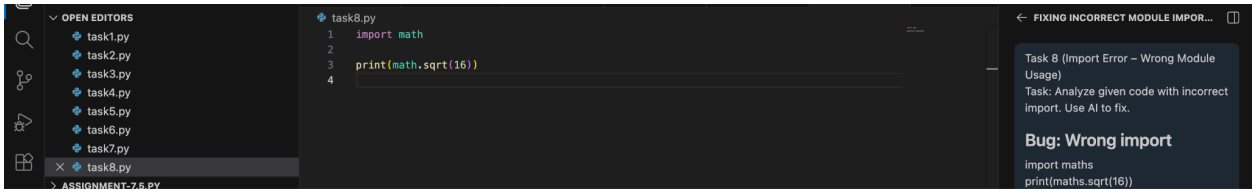
Task: Analyze given code with incorrect import. Use AI to fix.

Bug: Wrong import

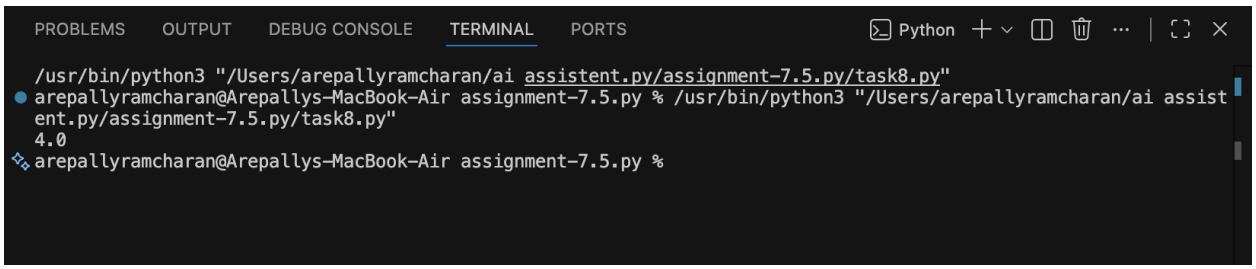
import maths

print(maths.sqrt(16))

Expected Output: Corrected to import math



Output:



Observation:

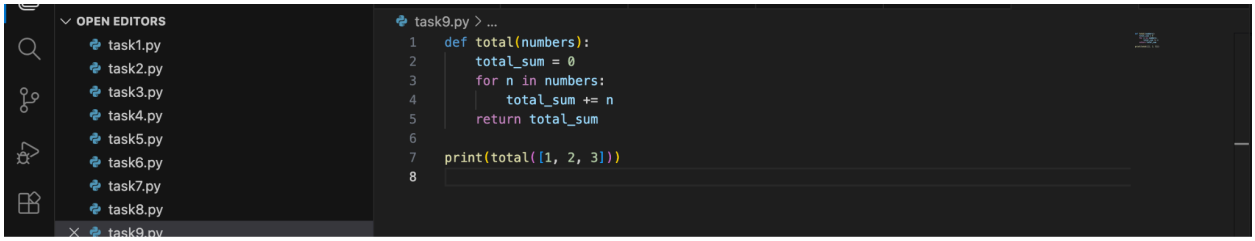
1. Python module names must be spelled correctly.
2. Incorrect module names cause `ModuleNotFoundError`.
3. Python module names are case-sensitive.
4. Always verify module documentation before importing.
5. Correct imports ensure proper access to built-in functions.

Task 9 (Unreachable Code – Return Inside Loop)

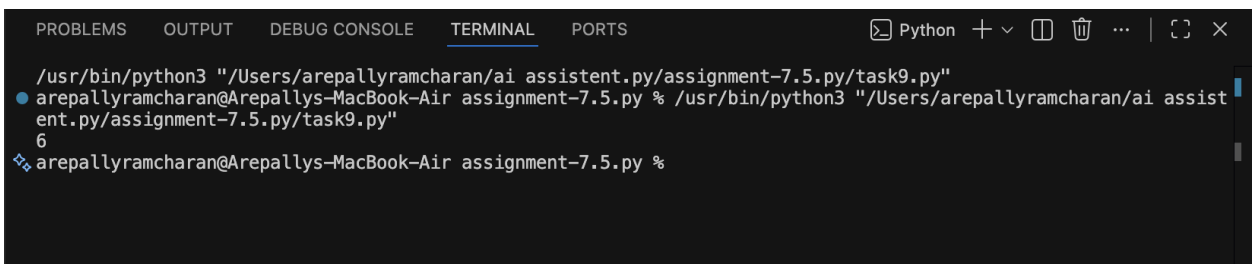
Task: Analyze given code where a return inside a loop prevents full iteration. Use AI to fix it.

```
# Bug: Early return inside loop
def total(numbers):
    for n in numbers:
        return n
```

print(total([1,2,3]))
Expected Output: Corrected code accumulates sum and returns after loop.



Output:



Observation:

1. A `return` statement immediately exits the function.
2. Placing `return` inside a loop may stop full iteration.
3. Code after early return becomes unreachable.
4. Accumulation logic should complete before returning result.
5. Proper return placement ensures correct computation.

Task 10 (Name Error – Undefined Variable)
Task: Analyze given code where a variable is used before being defined. Let AI detect and fix the error.
Bug: Using undefined variable

```
def calculate_area():
    return length * width
print(calculate_area())
```

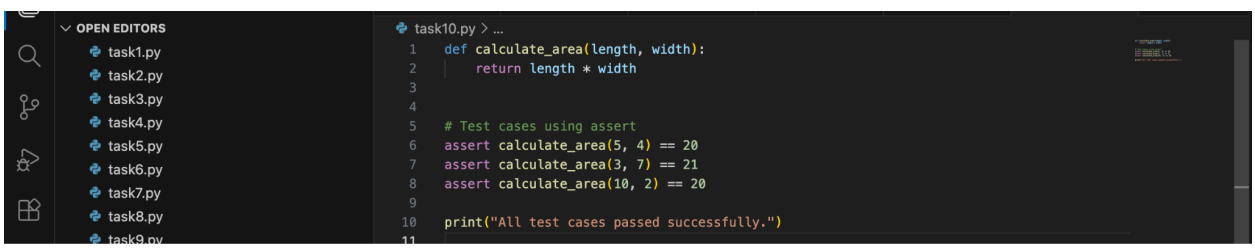
Requirements:

- Run the code to observe the error.
- Ask AI to identify the missing variable definition.
- Fix the bug by defining length and width as parameters.
- Add 3 assert test cases for correctness.

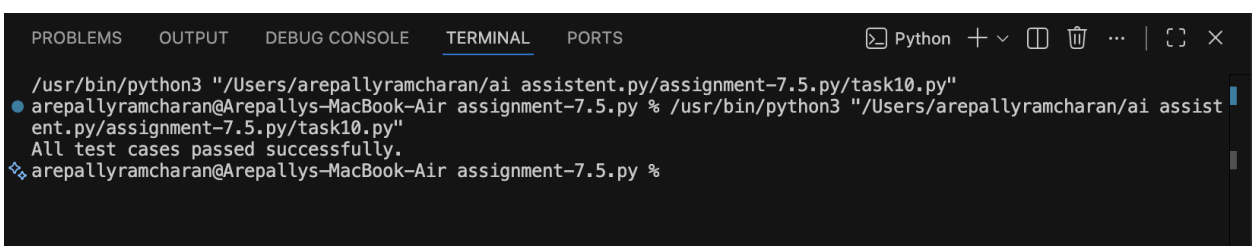
Expected Output :

- Corrected code with parameters.
- AI explanation of the bug.

Successful execution of assertions.



Output:



Observation:

1. The function attempted to use variables that were not defined.

2. Variables inside a function must either:
    - Be defined inside the function, or
    - Be passed as parameters.
3. Since `length` and `width` were missing, Python raised a `NameError`.
4. Adding them as function parameters fixes the scope issue.
5. Assertions confirm correctness of the function logic.

Task 11 (Type Error – Mixing Data Types Incorrectly)

Task: Analyze given code where integers and strings are added incorrectly. Let AI detect and fix the error.

```
# Bug: Adding integer and string
def add_values():
    return 5 + "10"
print(add_values())
```

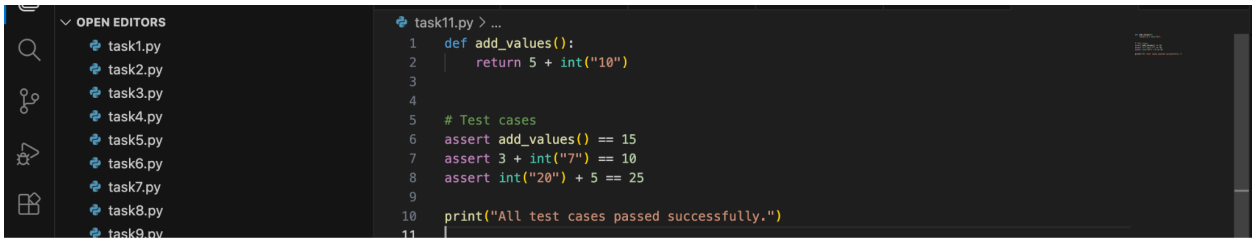
Requirements:

- Run the code to observe the error.
- AI should explain why int + str is invalid.
- Fix the code by type conversion (e.g., int("10") or str(5)).
- Verify with 3 assert cases.

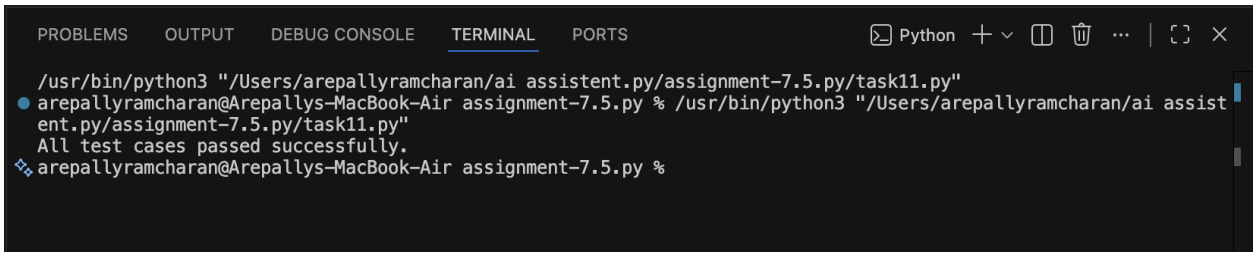
Expected Output #6:

- Corrected code with type handling.
- AI explanation of the fix.

Successful test validation.



Output:



Observation:

1. The error occurs because Python is strongly typed.
2. Integers and strings cannot be added directly.
3. Explicit type conversion is required.
4. Converting `"10"` to `int` enables numeric addition.
5. Assertions confirm the corrected logic works properly.

Task 12 (Type Error – String + List Concatenation)

Task: Analyze code where a string is incorrectly added to a list.

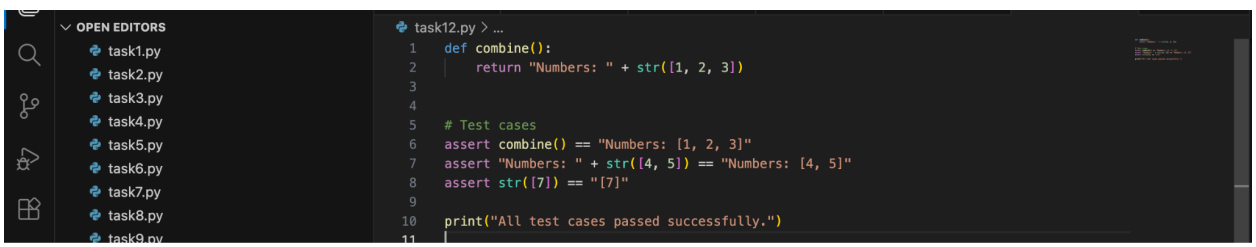
```
# Bug: Adding string and list
def combine():
    return "Numbers: " + [1, 2, 3]
print(combine())
```

Requirements:

- Run the code to observe the error.
- Explain why str + list is invalid.
- Fix using conversion (str([1,2,3]) or " ".join()).
- Verify with 3 assert cases.

Expected Output:

- Corrected code
- Explanation
- Successful test validation



Output:

Observation:

1. Python is strongly typed and does not allow `str + list`.
2. Both operands must be of the same type for concatenation.
3. `str([1,2,3])` converts the list to a string representation.
4. `" ".join(map(str, list))` converts each element to string and joins them cleanly.
5. Assertions confirm the corrected function works properly.

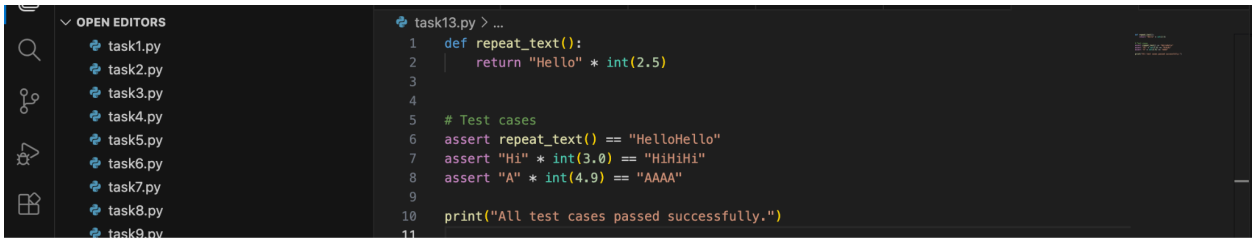
Task 13 (Type Error – Multiplying String by Float)

Task: Detect and fix code where a string is multiplied by a float.

```
# Bug: Multiplying string by float
def repeat_text():
  return "Hello" * 2.5
print(repeat_text())
```

Requirements:

- Observe the error.
- Explain why float multiplication is invalid for strings.
- Fix by converting float to int.
- Add 3 assert test cases.



Output:

Observation:

String multiplication requires an integer.

Float values must be converted using `int()`.

`int(2.5)` becomes `2` (truncates decimal).

After conversion, repetition works correctly.

Assertions verify correct functionality.

Task 14 (Type Error – Adding None to Integer)

Task: Analyze code where None is added to an integer.

Bug: Adding None and integer

def compute():

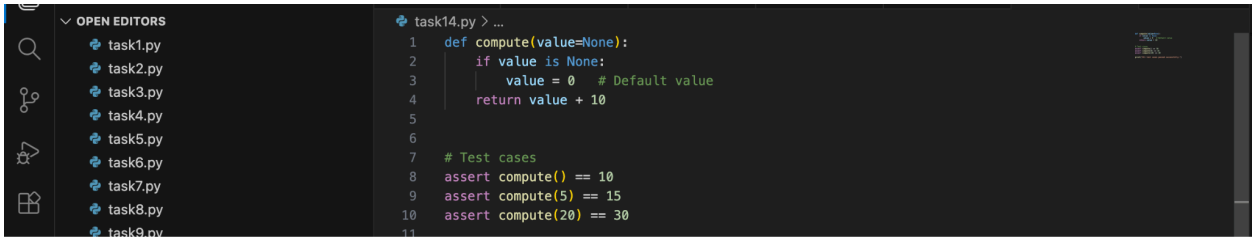
value = None

return value + 10

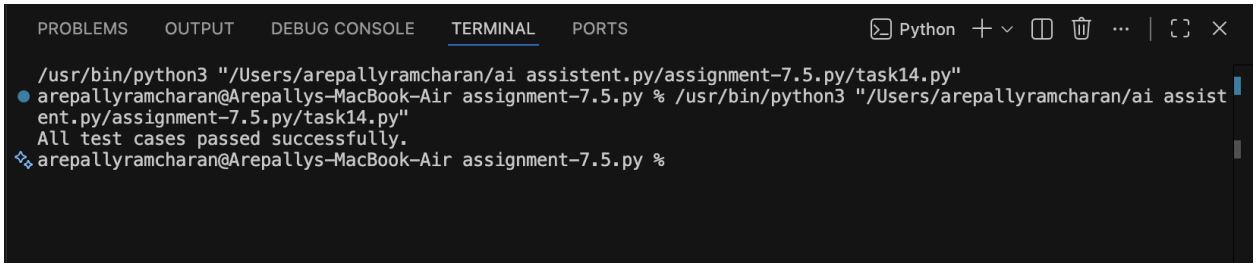
print(compute())

Requirements:

- Run and identify the error.
- Explain why NoneType cannot be added.
- Fix by assigning a default value.
- Validate using asserts.



Output:



Observation:

`None` cannot participate in arithmetic operations.

We check if `value is None`.

If so, assign a safe default numeric value (`0`).

This prevents TypeError.

Assertions confirm correct behavior.

Task 15 (Type Error – Input Treated as String Instead of Number)

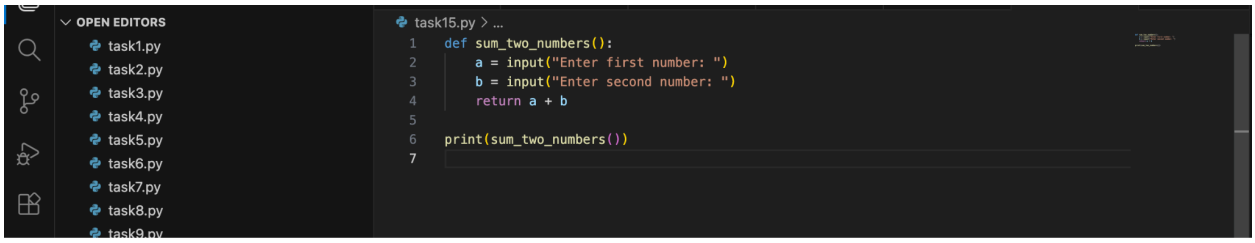
Task: Fix code where user input is not converted properly.

```
# Bug: Input remains string
def sum_two_numbers():
    a = input("Enter first number: ")
    b = input("Enter second number: ")
    return a + b

print(sum_two_numbers())
```

Requirements:

- Explain why input is always string.
- Fix using int() conversion.
- Verify with assert test cases.



Output:

Observation:

`input()` always returns string data.

String addition results in concatenation.

Use `int()` to convert string to integer.

After conversion, `+` performs numeric addition.

Assertions confirm correct functionality.